

仓颉编程语言实验报告

汇报人：南京大学 522025320035 高宇轩

实验时间：2025年12月

1. 代码结构与设计

1.1 整体架构

本次实验实现了一个完整的仓颉编程语言解释器，采用 **双系统设计**：静态类型检查系统（`typechecker.cj`）和动态运行时系统（`evaluator.cj`）。两个系统通过统一的数据结构和错误代码系统协同工作。

核心文件结构：

text

<code>typechecker.cj</code>	# 静态类型检查器
<code>statictype.cj</code>	# 静态类型系统定义
<code>ClassInfo.cj</code>	# 类信息存储结构
<code>evaluator.cj</code>	# 运行时解释器（包含执行引擎）

1.2 设计亮点

1.2.1 两阶段检查策略

为满足错误处理的特殊要求（先检查签名，后检查实现），我设计了 **两阶段检查策略**：

cj

```
// typechecker.cj
private func checkProgram(program: Program): Unit {
    // 第一阶段：收集所有声明的"签名"
    for (d in program.decls.iterator()) {
        collectDeclSignature(d) // 只收集，不检查体
    }

    // 第二阶段：检查声明体
    for (d in program.decls.iterator()) {
        checkDeclBody(d) // 检查函数体、初始化表达式
    }
}
```

这种设计使得：

- 可以提前发现类型不匹配错误（如示例2中的 `DEF_TYPE_MISMATCH`）
- 符合实验要求的错误报告顺序

- 避免了对未定义标识符的错误依赖

1.2.2 优雅的表达式处理

表达式处理采用 **访问者模式** 实现，每种表达式类型都有独立的处理方法：

cj

```
// evaluator.cj - 统一的表达式处理接口
public open override func visit(expr: BinaryExpr): Value {
    let left = expr.left.traverse(this)
    let right = expr.right.traverse(this)

    match (expr.oper.kind) {
        case TokenKind.ADD =>
            match ((left, right)) {
                case (VInteger(a), VInteger(b)) => value.from(a + b)
                case (VString(a), VString(b)) => value.from(a + b)
                case _ => throw TypeMismatchError(...)
            }
        // ... 其他操作符
    }
}
```

优点：

- 类型安全的模式匹配
- 清晰的错误处理路径
- 易于扩展新的表达式类型

1.2.3 作用域管理的创新设计

我设计了 **分层作用域系统**，支持复杂的变量访问规则：

cj

```
// 作用域栈设计
private var scopes = ArrayList<HashMap<String, VarInfo>>()

// 变量查找：从内向外搜索
private func findScope(name: String): ?HashMap<String, VarInfo> {
    var i = this.scopes.size - 1
    while (i >= 0) {
        let scope = this.scopes[i]
        if (scope.contains(name)) {
            return Some(scope)
        }
        i = i - 1
    }
    return None
}

// 闭包环境复制（关键）
private func copyScopesForClosure(): ArrayList<HashMap<String, VarInfo>> {
    let copied = ArrayList<HashMap<String, VarInfo>>()
    for (scope in this.scopes.iterator()) {
```

```

        let newScope = HashMap<String, VarInfo>()
        for ((k, v) in scope.iterator()) {
            newScope[k] = v // 浅拷贝变量引用
        }
        copied.add(newScope)
    }
    return copied
}

```

创新点：

- 支持函数闭包的正确实现
- 跟踪变量定义位置，实现 **非局部可变变量访问限制**
- 区分全局变量和局部变量的不同处理规则

1.2.4 类的统一处理框架

为了实现类系统的完整支持，我设计了 **三层结构**：

1. **静态类信息** (`Classes`)：存储类的结构模板
2. **类声明映射** (`ClassDeclMap`)：存储AST中的原始类定义
3. **对象实例** (`ObjectInstance`)：运行时的具体对象

cj

```

// 关键设计：分离静态信息和运行时实例
public class Classes {
    public let className: String
    public let classDecl: ClassDecl // 原始声明，用于方法查找
    public let fields = HashMap<String, VarInfo>() // 字段静态信息
}

public class ObjectInstance {
    public let className: String
    public let fields = HashMap<String, value>() // 运行时字段值
}

```

优势：

- 静态检查时使用 `Classes` 进行类型验证
- 运行时使用 `ObjectInstance` 存储实际值
- 通过 `classDecl` 实现方法动态查找

1.2.5 统一错误处理系统

cj

```

// 统一的错误代码和位置信息
public class CjcjRuntimeErrorWithLocation <: Exception {
    public let code: ErrorCode
    public let message: String
    public let location: ?Node

    public init(code: ErrorCode, message: String, location: ?Node) {

```

```

        this.code = code
        this.message = message
        this.location = location
    }
}

// 类型检查和运行时共享的错误代码
public enum ErrorCode {
    | UNDEFINED_VAR
    | DEF_TYPE_MISMATCH
    | ASSIGN_IMMUT_DECL // 修改为DECL，支持更广泛的不可变实体
    | THIS_OUTSIDE_CLASS
    | FIELD_INIT_TYPE_MISMATCH
    | MEMBER_NOT_INITIALIZED_AFTER_INIT
    // ... 共30+种错误类型
}

```

1.3 代码整洁性设计

1.3.1 模块化分离

- **数据定义**：value、StaticType 等核心数据结构单独定义
- **算法实现**：类型检查和运行时分离，职责清晰
- **辅助函数**：大量小型辅助函数，每个函数只做一件事

1.3.2 一致的命名规范

- 类名：PascalCase（如 ObjectInstance）
- 函数名：camelCase（如 findScope）
- 变量名：camelCase（如 currentClassName）
- 枚举值：UPPER_CASE（如 VInteger）

1.3.3 详细的注释和文档

- 每个关键函数都有目的说明
- 复杂算法有步骤注释
- 数据结构有字段说明

2. 遇到的问题与解决方案

2.1 主要问题及解决

问题1：类型检查与运行时类型不一致

问题描述：

在实现对象类型检查时，发现类型检查器使用 `StaticType.TClass("Rectangle")`，而运行时的 `valType` 方法返回 "Object"，导致类型匹配失败。

解决方案：

cj

```
// 修改 valType 方法
```

```
private fun valueType(v: Value): String {
    match (v) {
        case value.vObject(obj) => return obj.className // 返回类名而非"Object"
        case value.vInteger(_) => "Int64"
        // ... 其他类型
    }
}

// 同时修改类型检查逻辑
case value.vObject(obj) =>
    if (obj.className != declaredClassName) {
        throw CjcjRuntimeErrorWithLocation(
            ErrorCode.DEF_TYPE_MISMATCH,
            "Variable '${name}' declared as ${declaredClassName}, " +
            "but assigned object of class ${obj.className}",
            v
        )
    }
}
```

问题2：构造函数中的字段初始化检查

问题描述：

需要确保类的所有字段在构造函数执行后被初始化，但构造函数可能有多个分支路径。

解决方案：

```
// 使用字段初始化状态追踪
public class ClassInfo {
    public let fieldInitialized = HashMap<String, Bool>()

    // 在收集签名阶段记录字段初始化状态
    case v: VarDecl =>
        info.fields[v.identifier.value] = fromTypeNode(v.declType.getOrThrow())
        info.fieldInitialized[v.identifier.value] = v.initializer.isSome()
}

// 在构造函数检查时验证
if (isInit) {
    f.block.getOrThrow().traverse(this)

    // 检查所有字段是否被初始化
    for ((fieldName, initd) in info.fieldInitialized) {
        if (!initd) {
            throw CjcjRuntimeErrorWithLocation(
                ErrorCode.MEMBER_NOT_INITIALIZED_AFTER_INIT,
                "field `${fieldName}` is not initialized after init()",
                f
            )
        }
    }
}
}
```

问题3：方法调用时的 this 绑定

问题描述：

类方法需要访问对象的字段，但方法闭包创建时对象可能还不存在。

解决方案：

```
// 延迟 this 绑定
private func buildMethodFunction(f: FuncDecl, obj: ObjectInstance): UserFunction
{
    let captured = this.copyScopesForClosure()

    // 在最内层作用域添加 this 绑定
    let methodScope = HashMap<String, VarInfo>()
    methodScope["this"] = VarInfo(
        false,
        None,
        true,
        value.vObject(obj), // 绑定到具体对象
        false
    )
    captured.add(methodScope)

    return UserFunction(f.identifier.value, ..., captured, ...)
}

// 在方法调用时使用正确的作用域链
private func callUserFunction(fn: UserFunction, call: CallExpr): Value {
    let oldScopes = this.scopes
    this.scopes = fn.capturedScopes // 切换到方法的作用域链
    // ... 执行方法体
    this.scopes = oldScopes // 恢复
}
```

问题4：错误处理顺序要求

问题描述：

实验要求特定的错误报告顺序（先签名检查，后实现检查），与自然的AST遍历顺序冲突。

解决方案：

实现**两阶段遍历**：

1. **签名收集阶段**：遍历所有声明，只检查类型签名
2. **实现检查阶段**：再次遍历，检查函数体和初始化表达式

cj

```
// typechecker.cj 中的实现
private func checkProgram(program: Program): Unit {
  // 第一阶段
  for (d in program.decls.iterator()) {
    collectDeclSignature(d) // 不深入检查
  }

  // 第二阶段
  for (d in program.decls.iterator()) {
    checkDeclBody(d) // 详细检查
  }
}
```

2.2 实验难度与改进空间

实验难度评估：

1. **中等偏上**：需要深入理解类型系统、作用域、闭包等编译原理概念
2. **代码量大**：完整实现类型检查和运行时系统约2000+行代码
3. **调试困难**：运行时错误需要结合类型信息分析

可改进之处：

1. **性能优化**：
 - 当前类方法查找是线性搜索，可改用哈希表
 - 作用域复制可以优化为惰性复制
2. **错误信息**：
 - 可以添加更多上下文信息
 - 支持错误恢复和多错误报告
3. **功能扩展**：
 - 支持类继承和接口
 - 添加更多内置类型和函数
 - 支持模块系统

3. 已知 Bug

3.1 确认修复的Bug

1. **字段初始化顺序问题**：
 - **现象**：字段初始化表达式不能引用同一类中后面定义的字段
 - **状态**：已修复，通过两阶段检查避免
2. **构造函数返回类型检查**：
 - **现象**：构造函数声明返回类型时未报错
 - **状态**：已修复，增加专门检查

3.2 潜在未修复问题

1. **递归函数类型推断**：
 - 递归函数的类型推断可能不够完善

- 影响：深度递归可能无法正确推断类型
2. 复杂闭包环境的内存管理：
- 闭包环境复制可能造成内存使用较高
 - 影响：大规模闭包嵌套时性能可能下降
3. 边界情况下的整数溢出：
- 某些边界情况下的整数运算可能未正确处理溢出
 - 影响：极端数值计算可能产生意外结果

3.3 测试覆盖不足

当前测试主要覆盖：

- 基本类型操作
- 类的基本功能
- 常见错误情况

未充分测试：

- 大规模程序 (>1000行)
- 复杂的继承关系
- 并发访问 (虽然语言本身不支持并发)

总结

本次实验成功实现了仓颉编程语言的完整解释器，包括静态类型检查和动态运行时系统。通过创新的两阶段检查策略、统一的作用域管理系统和清晰的代码架构，实现了对类系统的完整支持。虽然在开发过程中遇到了类型一致性、作用域管理和错误处理顺序等多个挑战，但通过系统化的设计和细致的调试，最终实现了符合所有要求的解决方案。

实验不仅加深了对编译原理核心概念的理解，也锻炼了大型软件系统的设计和实现能力。代码具有良好的可读性和可维护性，为后续的功能扩展奠定了基础。